



PROGRAMA DE ASIGNATURA · NIVEL DE GRADO

Física Computacional I

FIS-2509

Distribución	HT	HP	HI	H-DE	Total
Semanales	1	4	0	9	14
Semestre	16	64	0	144	224

CRÉDITOS

3

Prerrequisitos: FIS-2112

Correquisitos: N/A

Elaborado y/o actualizado por: Vladimir Pérez, PhD., Nelphy de la Cruz, PhD., Erika Montero, MSc.

Fecha de última actualización: 2024-05-10

DESCRIPCIÓN DE LA ASIGNATURA

Física Computacional I introduce al estudiante en la programación científica como herramienta para modelar, simular y resolver problemas de la física y de áreas afines como las ciencias de la Tierra y la astronomía. Mediante los lenguajes C++ y Python, el curso desarrolla los fundamentos de la programación estructurada y orientada a objetos y su aplicación al análisis de fenómenos naturales y a la solución de problemas matemáticos y numéricos.

El recorrido inicia con la configuración de entornos de programación científica en Linux, incluyendo herramientas de desarrollo y control de versiones, y avanza hacia la programación en C++ (control de flujo, funciones, clases, herencia, polimorfismo y librería estándar) con énfasis en la gestión de archivos de datos científicos en formatos especializados. La parte final introduce Python y sus librerías científicas (NumPy, SciPy, Pandas, SymPy y Matplotlib) para la simulación, el procesamiento de grandes conjuntos de datos y la visualización de resultados.

Cada unidad se acompaña de estudios de caso y proyectos de programación tomados de contextos científicos reales, de modo que el estudiante no solo aprenda a programar, sino que comprenda cómo aplicar estas habilidades para resolver problemas prácticos y teóricos, sentando las bases de la física computacional que se profundiza en asignaturas posteriores.

PERFIL DEL/LOS DOCENTE(S)

El docente debe poseer, como mínimo, título de Maestría (preferiblemente Doctorado) en Física o áreas afines, con especialización o experiencia demostrable en física computacional y excelente dominio de los lenguajes C++ y Python y de los entornos de desarrollo en Linux. Se espera experiencia en la enseñanza universitaria de la programación científica con metodologías activas (aprendizaje basado en problemas y en proyectos, aula invertida), participación en investigación en el campo computacional, destreza en el uso de simulaciones numéricas para la demostración de conceptos físicos, compromiso con la actualización continua y capacidad para orientar a los estudiantes en sus proyectos y en su desarrollo académico.

COMPETENCIAS

Fundamentales

- CF2** Utilizar el razonamiento lógico, creativo, crítico y reflexivo en el análisis de la realidad desde perspectivas variadas, con el propósito de colaborar con la solución de problemas nacionales y globales.
- CF3** Utilizar los recursos tecnológicos disponibles en su área de estudio y desempeño, con la finalidad de realizar eficazmente sus labores en los ámbitos académicos, profesionales y personales.

Genéricas

- CG7** Asumir un comportamiento ético en todas sus actuaciones profesionales y personales para contribuir con la integridad de los procesos institucionales y sociales, la convivencia armoniosa con los demás seres humanos, y la protección del ambiente.

Específicas

- CE1** Analizar y aplicar los conceptos, principios, leyes y teorías de la Física, con sus alcances, limitaciones y procedimientos, para ofrecer explicaciones a los fenómenos naturales y encontrar soluciones a situaciones problemáticas.
- CE5** Utilizar simulaciones y una variedad de herramientas e instrumentos tecnológicos y computacionales para investigar y analizar fenómenos y problemas físicos, destacando en la aplicación de técnicas modernas del campo.
- CE7** Analizar fenómenos de manera colaborativa con pares de otras áreas del conocimiento, utilizando conceptos y métodos propios de las ciencias para resolver problemas y comunicar resultados.

RESULTADOS DE APRENDIZAJE ESPERADOS

- RAE-1** Configurar y administrar entornos de programación científica en Linux, integrando herramientas de desarrollo y de control de versiones.
- RAE-2** Desarrollar programas estructurados en C++ que implementen algoritmos de cálculo numérico para resolver problemas de la física.
- RAE-3** Diseñar clases y jerarquías orientadas a objetos en C++, aplicando herencia, polimorfismo y sobrecarga de operadores para modelar sistemas físicos y astronómicos.
- RAE-4** Procesar archivos de datos científicos en formatos comunes y especializados de la física, las geociencias y la astronomía mediante programas en C++.
- RAE-5** Aplicar los contenedores, iteradores y algoritmos de la librería estándar de C++ a la organización y el análisis de datos experimentales y observacionales.
- RAE-6** Desarrollar programas en Python que empleen sus librerías científicas para simular, procesar y visualizar fenómenos físicos, geocientíficos y astronómicos.
- RAE-7** Colaborar en el desarrollo de proyectos de software científico en equipo, utilizando control de versiones y comunicando los resultados con lenguaje científico.

CONTENIDO

Unidad 1: Linux y configuración del entorno de programación

Introducción a Linux y sus distribuciones; distribuciones utilizadas en centros de investigación y supercomputadoras; configuración del entorno de programación en Linux; estudios de caso: procesamiento de archivos de salida de simulaciones, búsqueda de archivos desde la consola y configuración y uso remoto de un servidor Linux

Unidad 2: Programación estructurada en C++ y aplicaciones

Introducción a la programación estructurada en C++; control de flujo; funciones; errores y excepciones; proyectos: evaluación de funciones matemáticas de la física, análisis del movimiento de una partícula, algoritmos de cálculo numérico, operaciones vectoriales y matriciales, y solución de ecuaciones algebraicas

Unidad 3: Programación orientada a objetos en C++

Introducción a la programación orientada a objetos; definición de clases y objetos; atributos y métodos; herencia y polimorfismo; constructores y destructores; sobrecarga de operadores; contenedores (arrays, vectores); proyectos: clases interoperables de vectores y matrices, simulación de un aparato mediante composición y herencia, base de datos de objetos del Sistema Solar e interfaz de usuario por consola

Unidad 4: Lectura y escritura de archivos en C++ para la gestión de datos científicos

Flujos de entrada y salida en C++; formatos de archivo comunes: CSV, TXT, XML, HTML y JSON; formatos especializados de la física y la astronomía (HDF5, NetCDF, FITS, GRIB), la química (CIF, MOL, SDF), las geociencias (GeoJSON, KML, SHP) y la biofísica (PDB, FASTA, GenBank); proyectos: conversión entre formatos especializados, descarga automatizada de archivos por identificador, estadística sobre archivos que no caben en memoria e interoperabilidad de datos entre plataformas científicas

Unidad 5: Librería estándar de C++ y aplicaciones

Introducción a la librería estándar de C++; contenedores e iteradores; algoritmos de la librería estándar; proyectos: comparación del rendimiento de operaciones sobre contenedores, organización y análisis de datos experimentales, procesamiento de cadenas en datos geocientíficos, almacenamiento de series temporales atmosféricas y astronómicas, y encriptación de archivos de texto

Unidad 6: Introducción a Python

Python y sus ventajas para la computación científica; instalación y configuración del entorno local y en línea; variables y tipos de datos; operadores y expresiones; control de flujo: condicionales, bucles y saltos; funciones; proyectos: reimplementación en Python de proyectos anteriores y acceso a bases de datos científicas abiertas en repositorios en línea

Unidad 7: Librerías de Python para la física, las ciencias de la Tierra y la astronomía

Librerías científicas de Python: NumPy, SciPy, Pandas, SymPy y Matplotlib; proyectos: resolución simbólica de integrales y ecuaciones diferenciales con SymPy, análisis estadístico de bases de datos en línea con SciPy, visualización de patrones climáticos, distribuciones geológicas y observaciones astronómicas con Matplotlib, procesamiento de series temporales con Pandas, y cálculo de valores y vectores propios con NumPy aplicado a problemas físicos

ESTRATEGIAS Y ACTIVIDADES FORMATIVAS

E.1	Clase expositiva del docente: Exposición organizada y rigurosa de los fundamentos teóricos de la unidad.
E.2	Aprendizaje basado en problemas (ABP): Resolución de problemas teóricos y aplicados que exigen integrar los conceptos de la unidad.
E.4	Aprendizaje basado en proyectos: Proyecto de investigación o desarrollo que culmina en informe y presentación.
E.10	Demostraciones y simulaciones en clase: Uso de demostraciones y simulaciones numéricas para visualizar conceptos abstractos.
E.18	Resolución de ejercicios y problemas: Práctica sistemática de ejercicios para consolidar procedimientos de cálculo y modelado.

EVALUACIÓN DE LOS APRENDIZAJES

Criterios

C.1	Nivel de dominio de los conceptos y procedimientos.
C.2	Aplicación de conceptos y procedimientos a la resolución de problemas.
C.3	Capacidad de análisis, comprensión y síntesis.
C.4	Habilidades de comunicación científica, oral y escrita.
C.5	Uso efectivo de herramientas computacionales y de simulación.

Técnicas e instrumentos

Técnica	Instrumentos	Criterios
TE.1 Pruebas escritas u orales	Exámenes parciales y final; Pruebas cortas (quizzes)	C.1 C.2
TE.3 Trabajos escritos, individuales o en grupo	Ejercicios y asignaciones; Proyectos (individuales o colaborativos)	C.2 C.3 C.5
TE.2 Exposiciones y presentaciones orales	Exposiciones y defensas orales	C.4

La evaluación es formativa, continua e integral (Res. 2021-227 y Res. 2003-084). Se aplican al menos dos evaluaciones parciales; ninguna prueba escrita puede exceder el 40 % de la puntuación total (Res. 72-346). La ponderación detallada se especifica en la guía didáctica de la asignatura.

RECURSOS DIDÁCTICOS Y TECNOLÓGICOS

Didácticos

- R1 Bibliografía académica: Libros de texto, manuales, artículos científicos y bases de datos.
- R2 Materiales de apoyo: Guías, cuadernos de laboratorio y colecciones de ejercicios.
- R3 Recursos digitales: Videos educativos, simulaciones interactivas, infografías y presentaciones.

Tecnológicos

- R7 Equipos digitales: Computadoras y dispositivos con acceso a internet.
- R8 Software especializado: Python, MATLAB, R, simuladores y entornos de desarrollo (Jupyter).
- R4 Plataformas educativas: Sistemas de gestión del aprendizaje (Moodle, Blackboard, edX).
- R5 Ambientes colaborativos: Herramientas de trabajo grupal y comunicación (Teams, Slack).
- R11 Conectividad en clases: Acceso a internet en el aula o espacio de enseñanza.

REFERENCIAS BIBLIOGRÁFICAS

Básicas

- Anagnostopoulos, K. N. (2016). Computational physics: A practical introduction to computational physics and scientific computing (using C++) (Vol. 1). Lulu Press.
- Landau, R. H., Páez, M. J., & Bordeianu, C. C. (2024). Computational physics: Problem solving with Python (4.^a ed.). John Wiley & Sons.
- Boudreau, J. F., & Swanson, E. S. (2017). Applied computational physics. Oxford University Press.

Complementarias

- Hubbard, J. R. (2021). Programming with C++ (3.^a ed., Schaum's Outline Series). McGraw-Hill Education.
- Stroustrup, B. (2013). The C++ programming language (4.^a ed.). Addison-Wesley.
- McKinney, W. (2022). Python for data analysis (3.^a ed.). O'Reilly Media.
- Shimobaba, T., Weng, J., Sakurai, T., Okada, N., Nishitsuji, T., Takada, N., Masuda, N., & Ito, T. (2012). Computational wave optics library for C++: CWO++ library. Computer Physics Communications, 183(5), 1124-1138.